

**Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки**

Лабораторна робота №1

з дисципліни

«Об'єктно-орієнтоване програмування»

на тему

«Знайомство із середовищем розробки та складання модульних проєктів програм»

Виконав:

студент групи ІМ-55

Лобода А.А

Номер в списку -13

Перевірів:

Рекечинський Д.О

Київ 2026

Завдання

1. Створити у обраному середовищі розробки проєкт з ім'ям Lab1.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. За потреби налагодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.

Завдання згідно варіанту (Ж = 13):

B1 = 13 mod 4 = 1: Вікно діалогу з повзунком горизонтального скролінгу (Horizontal Scroll Bar) та дві кнопки: [Так] і [Відміна]. Рухаючи повзунок скролінгу користувач вводить число у діапазоні від 1 до 100. Після натискування кнопки [Так] вибране число буде відображатися у головному вікні.

B2 = (13+1) mod 4 = 2: Два вікна діалогу. Спочатку з'являється перше, яке має дві кнопки: [Далі >] і [Відміна]. Якщо натиснути кнопку [Далі >], то воно закриється і з'явиться друге діалогове вікно, яке має кнопки: [< Назад], [Так] і [Відміна]. Якщо натиснути кнопку [< Назад], вікно закриється і перехід до першого вікна.

Для усіх варіантів необхідно створити два пункти меню — "Робота1" та "Робота2".

1. Потрібно, щоб при виборі пункту меню "Робота1" виконувалося щось згідно варіанту B1.
2. Запрограмувати також, щоб при виборі пункту меню "Робота2" виконувалося щось згідно варіанту B2.
3. Вихідний текст функції B1 запрограмувати в окремому модулі, а вихідний текст для функції B2 — у інших окремих модулях. Оскільки B2 = 2, проєкт складається з головного файлу lab.py та трьох модулів — module1.py (Робота1), module2.py (Робота2, крок 1), module3.py (Робота2, крок 2).
4. Callback-функції вікон діалогу не мають бути видимими зовні модулів. У кожному модулі внутрішні обробники подій позначені префіксом "_" (конвенція приватності в Python), назовні експортується лише функція open_dialog().
5. Якщо виводиться наступне вікно діалогу, то попереднє автоматично закривається (self._win.destroy()) перед викликом наступного кроку).
6. Імпорт коду одного модуля з іншого не використовується — кожен модуль імпортує лише tkinter та спільну допоміжну функцію з utils.py.

Файл [lab1.py](#)

```
import tkinter as tk

import module1
import module2
import module3
from utils import center_window

class MainApp:
    def __init__(self, root):
```

```

        self._root = root
        self._root.title("Лабораторна робота №1")

        self._result_label = tk.Label(self._root, text="Результат: -",
font=("Arial", 12))
        self._result_label.pack(padx=20, pady=20)

        self._build_menu()
        center_window(self._root, 400, 150)

    def _build_menu(self):
        menu_bar = tk.Menu(self._root)
        menu_bar.add_command(label="Робота1",
command=self._open_robota1)
        menu_bar.add_command(label="Робота2",
command=self._open_robota2)
        self._root.config(menu=menu_bar)

    def _open_robota1(self):
        module1.open_dialog(self._root, self._on_robota1_result)

    def _on_robota1_result(self, value):
        self._result_label.config(text=f"Робота1: вибрано число
{value}")

    def _open_robota2(self):
        module2.open_dialog(self._root, self._open_robota2_step2)

    def _open_robota2_step2(self):
        module3.open_dialog(self._root, self._open_robota2)

if __name__ == "__main__":
    root = tk.Tk()
    app = MainApp(root)
    root.mainloop()

```

Файл [module1.py](#)

```

import tkinter as tk
from utils import center_window

class _Dialog1:

```

```
def __init__(self, parent, on_result):
    self._on_result = on_result
    self._value = 1

    self._win = tk.Toplevel(parent)
    self._win.title("Робота 1")
    self._win.resizable(False, False)
    self._win.grab_set()

    tk.Label(self._win, text="Виберіть число від 1 до
100:").pack(padx=20, pady=(15, 5))

    self._scale = tk.Scale(
        self._win, from_=1, to=100, orient="horizontal",
        length=250, command=self._on_scale_change
    )
    self._scale.set(1)
    self._scale.pack(padx=20, pady=5)

    btn_frame = tk.Frame(self._win)
    btn_frame.pack(pady=15)
    tk.Button(btn_frame, text="Так", width=10,
command=self._on_ok).pack(side="left", padx=5)
    tk.Button(btn_frame, text="Відміна", width=10,
command=self._on_cancel).pack(side="left", padx=5)

    center_window(self._win, 320, 160)
    self._win.protocol("WM_DELETE_WINDOW", self._on_cancel)

def _on_scale_change(self, value):
    self._value = int(value)

def _on_ok(self):
    self._on_result(self._value)
    self._win.destroy()

def _on_cancel(self):
    self._win.destroy()

def open_dialog(parent, on_result):
    _Dialog1(parent, on_result)
```

Файл [module2.py](#)

```
import tkinter as tk
from utils import center_window

class _Step1:
    def __init__(self, parent, on_next):
        self._on_next = on_next

        self._win = tk.Toplevel(parent)
        self._win.title("Робота 2 - Крок 1")
        self._win.resizable(False, False)
        self._win.grab_set()

        tk.Label(self._win, text="Перше вікно діалогу.").pack(padx=20,
pady=20)

        btn_frame = tk.Frame(self._win)
        btn_frame.pack(pady=10)
        tk.Button(btn_frame, text="Далі >", width=10,
command=self._on_go_next).pack(side="left", padx=5)
        tk.Button(btn_frame, text="Відміна", width=10,
command=self._on_cancel).pack(side="left", padx=5)

        center_window(self._win, 300, 130)
        self._win.protocol("WM_DELETE_WINDOW", self._on_cancel)

    def _on_go_next(self):
        self._win.destroy()
        self._on_next()

    def _on_cancel(self):
        self._win.destroy()

def open_dialog(parent, on_next):
    _Step1(parent, on_next)
```

Файл [module3.py](#)

```
import tkinter as tk
from utils import center_window
```

```

class _Step2:
    def __init__(self, parent, on_back):
        self._on_back = on_back

        self._win = tk.Toplevel(parent)
        self._win.title("Робота 2 - Крок 2")
        self._win.resizable(False, False)
        self._win.grab_set()

        tk.Label(self._win, text="Друге вікно діалогу.").pack(padx=20,
pady=20)

        btn_frame = tk.Frame(self._win)
        btn_frame.pack(pady=10)
        tk.Button(btn_frame, text="< Назад", width=10,
command=self._on_go_back).pack(side="left", padx=5)
        tk.Button(btn_frame, text="Так", width=10,
command=self._on_finish).pack(side="left", padx=5)
        tk.Button(btn_frame, text="Відміна", width=10,
command=self._on_cancel).pack(side="left", padx=5)

        center_window(self._win, 340, 130)
        self._win.protocol("WM_DELETE_WINDOW", self._on_cancel)

    def _on_go_back(self):
        self._win.destroy()
        self._on_back()

    def _on_finish(self):
        self._win.destroy()

    def _on_cancel(self):
        self._win.destroy()

def open_dialog(parent, on_back):
    _Step2(parent, on_back)

```

Файл [utils.py](#)

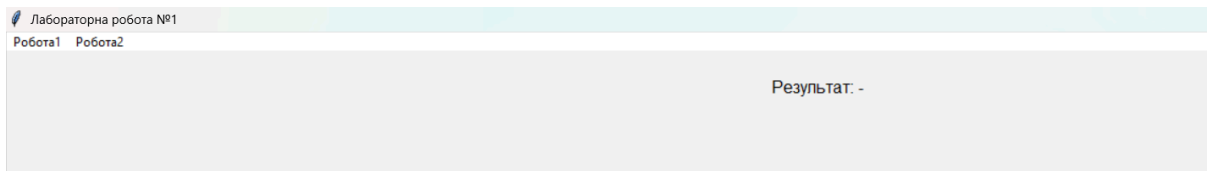
```

def center_window(win, width, height):

```

```
win.update_idletasks()
screen_width = win.winfo_screenwidth()
screen_height = win.winfo_screenheight()
x = (screen_width - width) // 2
y = (screen_height - height) // 2
win.geometry(f"{width}x{height}+{x}+{y}")
```

Скріншоти:



Результат: -

Робота 1

Виберіть число від 1 до 100:

1

Так Відміна

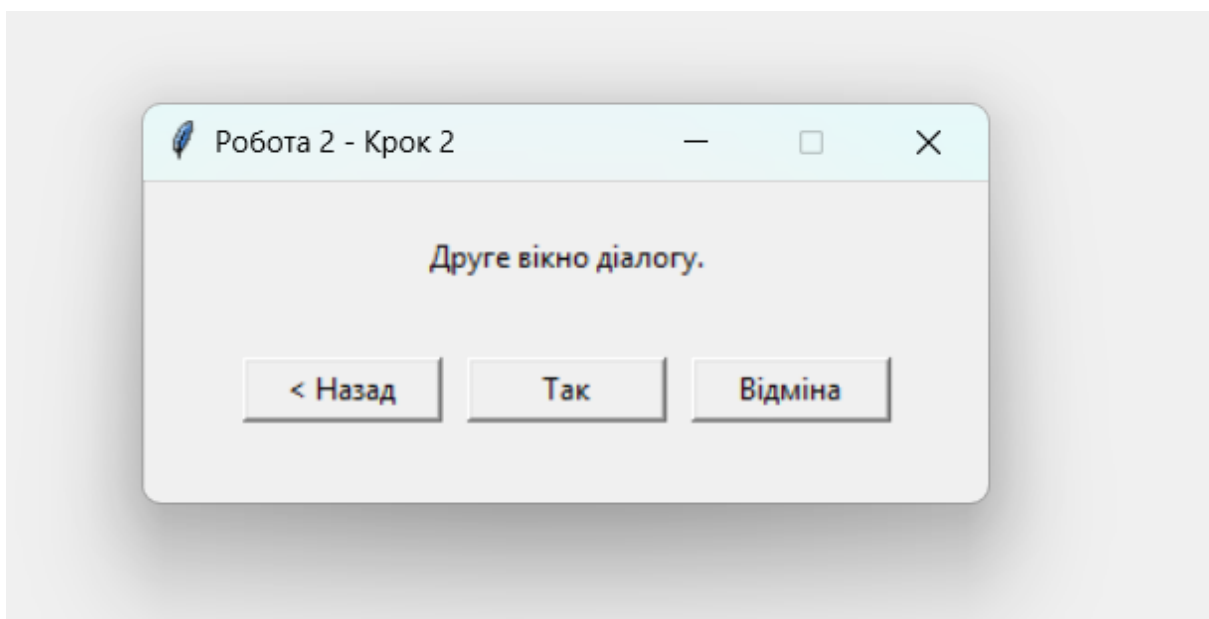
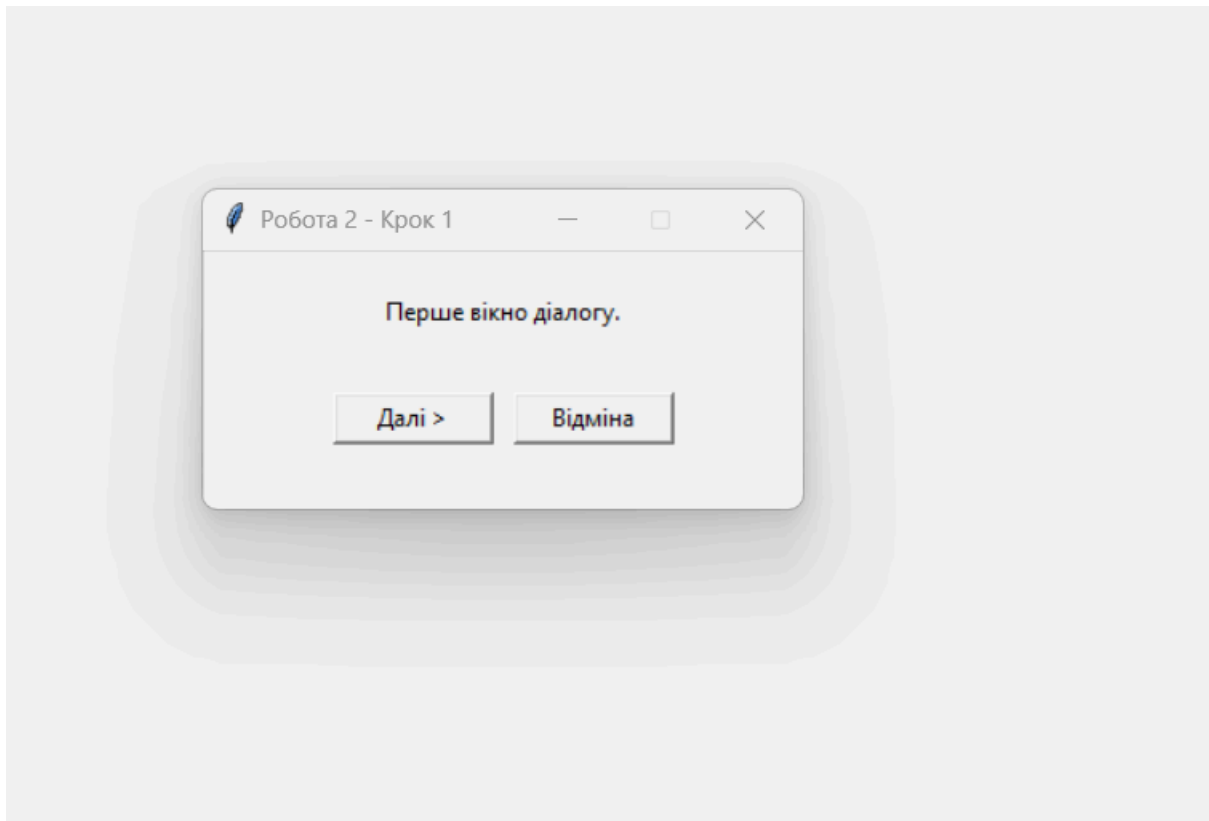
Результат: -

Робота 1

Виберіть число від 1 до 100:

48

Так Відміна



Висновок: Під час виконання лабораторної роботи №1 було створено багатовіконний застосунок на мові Python з використанням бібліотеки Tkinter та модульної архітектури.

- Реалізовано два пункти головного меню — "Робота1" та "Робота2" — відповідно до вимог завдання.
- Функцію для варіанту $B1 = 1$ ($Ж \bmod 4 = 13 \bmod 4 = 1$) — вікно діалогу з повзунком горизонтального скролінгу (від 1 до 100) — запрограмовано в окремому модулі module1.py.

- Функцію для варіанту $B2 = 2$ ($J+1 \bmod 4 = 14 \bmod 4 = 2$) — покроковий майстер із двох діалогових вікон — реалізовано у двох окремих модулях: module2.py (перший крок, кнопки "Далі >" і "Відміна") та module3.py (другий крок, кнопки "< Назад", "Так" і "Відміна"), оскільки кожне вікно діалогу є незалежним модулем.
- Забезпечено принципи модульності та інкапсуляції: усі внутрішні callback-методи класів діалогів приховані від зовнішнього доступу за допомогою конвенції приватності Python (префікс "_"), а назовні кожен модуль експортує лише одну функцію — open_dialog().
- Реалізовано автоматичне закриття попереднього діалогового вікна при переході до наступного та коректний зворотний перехід (з другого кроку Робота2 на перший).
- Спільну допоміжну логіку (центрування вікна на екрані) винесено в окремий файл utils.py, щоб уникнути дублювання коду між модулями — жоден модуль не імпортує код з іншого модуля напрямку.
- Отримано практичні навички роботи з подієво-орієнтованим програмуванням у Tkinter: обробка подій через command-callback, створення модальних вікон (Toplevel + grab_set), що є функціональним аналогом обробки повідомлень WM_COMMAND та циклу очікування повідомлень у Windows API.